# SESIÓN DE CLASE — Django REST Framework + API (FastAPI) + Proyecto Integrador

*Sesión N° 2 — 5 Octubre de 2026 (Lunes) — Bloque 1: Django REST Framework*

| | | | |
|---|---|---|---|
| **Instructor** | César Augusto Moreno Mena | **Centro** | CESGE |
| **Curso** | Django REST Framework + API (FastAPI) + Proyecto Integrador | **Año** | 2026 |

| N° | Fecha | Día | Semana | Bloque | Tema de la sesión | Teoría | Ejemplo | Práctica |
|---|---|---|---|---|---|---|---|---|
| 2 | 5 Octubre | Lunes | S2 | Bloque 1: Django REST Framework | **Modelos y ORM de Django** | Definición de modelos, tipos de campos, migraciones (makemigrations/migrate), consultas con el ORM. | Creación de modelos para un dominio de negocio (ej. productos, categorías) y generación de migraciones. | Definición del modelo de datos del proyecto Django personal. |

## 1. Objetivo de la sesión

Al finalizar esta sesión, el aprendiz será capaz de **definir modelos de datos en Django**, entender los **tipos de campo** más usados, **generar y aplicar migraciones**, y **consultar datos usando el ORM** desde la consola interactiva de Django.

## 2. ¿Qué es un modelo y qué es el ORM?

Un **modelo** es una clase de Python que representa una **tabla** de la base de datos. Cada atributo de la clase representa una **columna** de esa tabla, y cada instancia de la clase representa una **fila** (un registro).

El **ORM** (Object-Relational Mapper, "mapeador objeto-relacional") es el "traductor" que convierte tu código Python en instrucciones SQL, sin que tengas que escribir SQL a mano. Por ejemplo, cuando escribes `Producto.objects.all()`, Django traduce eso a `SELECT * FROM producto;` por ti.

> **Analogía**
> Piensa en el ORM como un traductor simultáneo en una reunión internacional: tú hablas Python, la base de datos solo entiende SQL, y el ORM traduce en ambos sentidos sin que tengas que aprender el otro idioma.

## 3. Tipos de campo más comunes

| | |
|---|---|
| **Aa** | `CharField(max_length=100)`: texto corto (nombre, título). **Siempre requiere** `max_length`. |
| ¶ | `TextField()`: texto largo, sin límite fijo (descripciones, comentarios). |
| # | `IntegerField()`: números enteros (stock, cantidad). |
| **$** | `DecimalField(max_digits=8, decimal_places=2)`: números decimales exactos, ideal para **precios y dinero** (nunca uses `FloatField` para dinero, porque puede generar errores de redondeo). |
| ✓ | `BooleanField(default=True)`: verdadero/falso (¿está activo?, ¿está disponible?). |
| □ | `DateField()` / `DateTimeField()`: fechas y fechas con hora. |
| □ | `ForeignKey(OtroModelo, on_delete=models.CASCADE)`: relación "muchos a uno" con otro modelo (ej. muchos productos pertenecen a una categoría). |

## 4. Ejemplo guiado: creación de modelos paso a paso

### Paso 1: Definir el modelo Categoria

Abre `inventario/models.py` y escribe:

```
from django.db import models

class Categoria(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(blank=True)

    def __str__(self):
        return self.nombre
```

> □**Idea clave**
> El método `__str__` define cómo se ve el objeto cuando se imprime o se muestra en el panel de administración. **Siempre** agrégalo a tus modelos; sin él, verás cosas confusas como `Categoria object (1)` en vez del nombre real.

### Paso 2: Definir el modelo Producto, relacionado con Categoria

```
class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    precio = models.DecimalField(max_digits=8, decimal_places=2)
    stock = models.IntegerField(default=0)
    disponible = models.BooleanField(default=True)
    categoria = models.ForeignKey(
        Categoria,
        on_delete=models.CASCADE,
        related_name='productos'
    )
```

```
    def __str__(self):
        return self.nombre
```

`on_delete=models.CASCADE` significa: "si se elimina la categoría, elimina también todos los productos asociados a ella". Es la opción más común para empezar.

### Paso 3: Generar las migraciones

Una **migración** es un archivo que describe los cambios que hiciste en tus modelos, para que Django los aplique a la base de datos. Es como un "historial de cambios" (similar a un commit de Git, pero para la estructura de la base de datos).

```
python manage.py makemigrations
```

Verás un mensaje como:

```
Migrations for 'inventario':
  inventario/migrations/0001_initial.py
    - Create model Categoria
    - Create model Producto
```

### Paso 4: Aplicar las migraciones

```
python manage.py migrate
```

Este comando toma el archivo de migración generado y **crea las tablas reales** en la base de datos (por defecto, Django usa SQLite, un archivo `db.sqlite3` que ya trae listo, ideal para aprender).

> ⚠ **Regla de oro**
> Cada vez que **agregues, elimines o modifiques** un campo en `models.py`, debes repetir **ambos** comandos en orden: primero `makemigrations`, luego `migrate`. Si solo haces uno de los dos, la base de datos y tu código quedarán desincronizados.

## 5. Consultas con el ORM (Django shell)

Django trae una consola interactiva especial para probar el ORM sin necesidad de crear vistas.

```
python manage.py shell
```

### Crear registros

```
>>> from inventario.models import Categoria, Producto
>>> cat = Categoria.objects.create(nombre="Tecnología")
>>> Producto.objects.create(nombre="Mouse inalámbrico", precio=45000, stock=20,
categoria=cat)
>>> Producto.objects.create(nombre="Teclado mecánico", precio=120000, stock=10,
categoria=cat)
```

### Consultar registros

```
>>> Producto.objects.all()                        # todos los productos
>>> Producto.objects.get(id=1)                     # un producto específico por id
>>> Producto.objects.filter(disponible=True)       # solo los disponibles
>>> Producto.objects.filter(precio__gte=50000)     # precio mayor o igual a 50000
>>> Producto.objects.filter(categoria__nombre="Tecnología")  # por relación
```

`__gte` significa "mayor o igual que" (greater than or equal). Existen otros similares: `__lte` (menor o igual), `__contains` (contiene un texto), `__icontains` (igual, pero sin distinguir mayúsculas).

## 6. Ejercicio práctico (para hacer en clase)

En tu propio proyecto (el que creaste el 2 de octubre), realiza lo siguiente paso a paso:

1. Define un modelo `Categoria` con al menos un campo `nombre`.
2. Define un modelo principal (por ejemplo `Producto`, `Libro` o el que corresponda a tu proyecto personal) con al menos **4 campos** de tipos distintos y una `ForeignKey` hacia `Categoria`.
3. Agrega el método `__str__` en ambos modelos.
4. Ejecuta `makemigrations` y luego `migrate`.
5. Abre el shell de Django y crea al menos **2 categorías** y **3 registros** del modelo principal, distribuidos entre esas categorías.
6. Desde el shell, realiza una consulta con `.filter()` y otra con `.get()`, y verifica que el resultado sea el esperado.

## 7. Errores comunes y cómo evitarlos

| | |
|---|---|
| ✗ | **"no such table"**: olvidaste ejecutar `migrate` después de `makemigrations`. |
| ✗ | **"NOT NULL constraint failed"**: intentaste guardar un registro sin llenar un campo obligatorio. Agrega `blank=True` (para formularios) o `null=True` (para la base de datos) si el campo puede quedar vacío, o simplemente llena el valor al crear el registro. |
| ✗ | **Olvidar `on_delete` en un ForeignKey**: Django lo exige siempre; sin él, obtendrás un error al hacer `makemigrations`. |
| ✗ | **Usar `FloatField` para dinero**: puede generar valores como `45000.000000001`. Usa siempre `DecimalField` para precios. |

## 8. Resumen de la sesión (checklist de aprendizaje)

| | |
|---|---|
| ☐ | Entiendo qué es un modelo y qué es el ORM. |
| ☐ | Conozco al menos 6 tipos de campo y cuándo usar cada uno. |
| ☐ | Sé crear una relación `ForeignKey` entre dos modelos. |
| ☐ | Sé generar y aplicar migraciones correctamente. |

| | |
|---|---|
| ☐ | Sé crear y consultar registros desde el shell de Django usando `.create()`, `.all()`, `.filter()` y `.get()`. |

*Próxima sesión (7 de octubre): usaremos el Django Admin para gestionar visualmente los datos que hoy aprendimos a modelar.*